Applying a function to all the values in a column apply()

Here function foo is applied to all rows

Creating a new column D by adding existing columns B and C

```
import pandas as pd
df = pd.DataFrame({"A":[2,3],"B":[4,5],"C":[6,7]})
print('Original Dataframe')
print(df)
def foo(row):
    return row["B"] + row["C"]
df["D"] = df.apply(foo, axis=1)
print('Modified Dataframe')
print(df)
```

Output:

```
Original Dataframe
   A  B  C
0  2  4  6
1  3  5  7
Modified Dataframe
   A  B  C   D
0  2  4  6  10
1  3  5  7  12
```

Creating a new column D by multiplying 2 with column A

```
import pandas as pd
df = pd.DataFrame({"A":[2,3],"B":[4,5],"C":[6,7]})
print('Original Dataframe')
print(df)
def foo(row):
    return(row*2)
df["D"] = df["A"].apply(foo)
print('Modified Dataframe')
print(df)
```

Output:

```
Original Dataframe
   A  B  C
0  2  4  6
1  3  5  7
Modified Dataframe
   A  B  C  D
0  2  4  6  4
1  3  5  7  6
```

Apply and lambda

Here a function is applied to all rows using lambda. If the function is simple lambda will be more faster.

Creating a new column D by adding existing columns B and C

```
import pandas as pd
df = pd.DataFrame({"A":[2,3],"B":[4,5],"C":[6,7]})
print('Original Dataframe')
print(df)
df["D"] = df.apply(lambda row: row["B"] + row["C"], axis=1)
print('Modified Dataframe')
print(df)
```

Output:

```
Original Dataframe
   A  B  C
0  2  4  6
1  3  5  7
Modified Dataframe
   A  B  C   D
0  2  4  6  10
1  3  5  7  12
```

Creating a new column D by multiplying 2 with column A

```
import pandas as pd
df = pd.DataFrame({"A":[2,3],"B":[4,5],"C":[6,7]})
print('Orignal Dataframe')
print(df)
df["D"] = df["A"].apply(lambda row: row*2)
print('Modified Dataframe')
print(df)
```

Output:

```
Orignal Dataframe
   A  B  C
0  2  4  6
1  3  5  7
Modified Dataframe
   A  B  C  D
0  2  4  6  4
1  3  5  7  6
```

Pandas Reading different file sources

To read files into a pandas DataFrame, you can use various functions based on the type of file you're working with (e.g., CSV, Excel, JSON, etc.).

1. Reading a CSV File

Use **pd.read_csv()** to read a CSV file.

```
import pandas as pd
# Reading a CSV file into a DataFrame in different styles
df=pd.read_csv('C:\\DataScience\\Workspace\\dataset\\data.csv')
#df=pd.read_csv('C:/DataScience/Workspace/dataset/data.csv')
#df = pd.read_csv(r'C:\DataScience\Workspace\dataset\data.csv')
# if current location is #'F:\DataScience\Workspace\
#df = pd.read_csv('dataset\data.csv') or
#df = pd.read_csv('dataset/data.csv')
if current location is dataset ie same folder where data is located
#df = pd.read_csv('data.csv')
```

```
# Display the first few 5 of the DataFrame
print(df.head())
# Display the first few 10 of the DataFrame
print(df.head(10))
```

2. Reading an Excel File

Use **pd.read_excel()** to read an Excel file. You can specify the sheet name or use the default first sheet.

```
import pandas as pd
# Reading an Excel file into a DataFrame
df = pd.read_excel('file.xls', sheet_name='Sheet1')
#pip install openpyxl
df = pd.read_excel('file.xlsx',engine='openpyxl') # only for xlsx \
files
# Display the first few rows of the DataFrame
print(df.head())
```

3. Reading a JSON File

Big data sets are often stored, or extracted as JSON. JSON is plain text, but has the format of an object, and is well known in the world of programming, including Pandas.

Use **pd.read_json()** to read a JSON file into a DataFrame.

```
import pandas as pd
# Reading a JSON file into a DataFrame
df = pd.read_json('file.json')

# Display the first few rows of the DataFrame
print(df.head())
```

Handling Encoding Issues

Sometimes, files may have encoding issues and data cant be retrieved. You can specify the encoding type (e.g., 'utf-8', 'latin1') when reading the file.

```
import pandas as pd
df = pd.read_csv('file.csv', encoding='utf-8')
```

encoding can be 'ascii', 'big5', 'big5hkscs', 'cp037', 'cp273', 'cp424', 'cp437', 'cp500', 'cp720', 'cp737', 'cp775', 'cp850', 'cp852', 'cp855', 'cp856', 'cp857', 'cp858', 'cp860', 'cp861', 'cp862', 'cp863', 'cp864', 'cp865', 'cp866', 'cp869', 'cp874', 'cp875', 'cp932', 'cp949', 'cp950' , 'cp1006', 'cp1026', 'cp1125', 'cp1140', 'cp1250', 'cp1251', 'cp1252', 'cp1253', 'cp1254', 'cp1255', 'cp1256', 'cp1257', 'cp1258', 'euc_jp', 'euc_jis_2004', 'euc_jisx0213', 'euc_kr', 'gb2312', 'gbk', 'gb18030', 'hz', 'iso2022_jp', 'iso2022_jp_1', 'iso2022_jp_2' , 'iso2022_jp_2004', 'iso2022_jp_3', 'iso2022_jp_ext', 'iso2022_kr', 'latin_1', 'iso8859_2' , 'iso8859_3', 'iso8859_4', 'iso8859_5', 'iso8859_6', 'iso8859_7', 'iso8859_8', 'iso8859_9' , 'iso8859_10', 'iso8859_11', 'iso8859_13', 'iso8859_14', 'iso8859_15', 'iso8859_16', 'johab' , 'koi8_r', 'koi8_t', 'koi8_u', 'kz1048', 'mac_cyrillic', 'mac_greek', 'mac_iceland', 'mac_latin2', 'mac_roman', 'mac_turkish', 'ptcp154', 'shift_jis', 'shift_jis_2004', 'shift_jisx0213', 'utf_32' , 'utf_32_be', 'utf_32_le', 'utf_16', 'utf_16_be', 'utf_16_le', 'utf_7', 'utf_8', 'utf_8_sig'.

For easy use just use the code, if encoding is unknown

```
import pandas as pd
```

```
encoding_list  =  ['ascii',  'big5',  'big5hkscs',  'cp037',  'cp273',\
'cp424', 'cp437', 'cp500', 'cp720', 'cp737', 'cp775', 'cp850', 'cp852',\
'cp855', 'cp856', 'cp857', 'cp858', 'cp860', 'cp861', 'cp862', 'cp863',\
'cp864', 'cp865', 'cp866', 'cp869', 'cp874', 'cp875', 'cp932', 'cp949',\
'cp950', 'cp1006', 'cp1026', 'cp1125', 'cp1140', 'cp1250', 'cp1251',\
'cp1252', 'cp1253', 'cp1254', 'cp1255', 'cp1256', 'cp1257', 'cp1258',\
'euc_jp', 'euc_jis_2004', 'euc_jisx0213', 'euc_kr', 'gb2312', 'gbk',\
'gb18030',   'hz',   'iso2022_jp',   'iso2022_jp_1',   'iso2022_jp_2',\
'iso2022_jp_2004',  'iso2022_jp_3',  'iso2022_jp_ext',  'iso2022_kr',\
'latin_1',   'iso8859_2',   'iso8859_3',   'iso8859_4',   'iso8859_5',\
'iso8859_6',  'iso8859_7',  'iso8859_8',  'iso8859_9',  'iso8859_10',\
'iso8859_11', 'iso8859_13', 'iso8859_14', 'iso8859_15', 'iso8859_16',\
'johab',  'koi8_r',  'koi8_t',  'koi8_u',  'kz1048',  'mac_cyrillic',\
'mac_greek', 'mac_iceland', 'mac_latin2', 'mac_roman', 'mac_turkish',\
'ptcp154', 'shift_jis', 'shift_jis_2004', 'shift_jisx0213', 'utf_32',\
'utf_32_be', 'utf_32_le', 'utf_16', 'utf_16_be', 'utf_16_le', 'utf_7',\
'utf_8', 'utf_8_sig']
for encoding in encoding_list:
      worked = True
      try:
         df = pd.read_csv(file.csv', encoding=encoding,  nrows=5)
      except:
         worked = False
      if worked:
            print(encoding, ':\n', df.head())
```

4. Reading from mysql database

To read a MySQL database into a Pandas DataFrame, you need to use the pandas library in combination with a database connector like mysql-connector

```
import mysql.connector
mydb = mysql.connector.connect(
  host="localhost",
  user="root",
  password="root",
  database="world"
)
import pandas as pd
query="select * from city limit 5"

df1 = pd.read_sql(query, mydb)
df2=df1[['ID','Name','CountryCode','District']]
print('Original Data')
print(df1)
print('Data with column filtered')
print(df2)
```

Output:

```
ID            Name      CountryCode      District  Population
0   1           Kabul         AFG          Kabol     1780000
1   2        Qandahar         AFG       Qandahar      237500
2   3           Herat         AFG          Herat      186800
3   4  Mazar-e-Sharif         AFG          Balkh      127800
4   5  Amsterdam              NLD   Noord-Holland      731200
   ID            Name    CountryCode     District
```

```
0   1           Kabul         AFG          Kabol
1   2         Qandahar        AFG        Qandahar
2   3           Herat         AFG          Herat
3   4  Mazar-e-Sharif         AFG          Balkh
4   5       Amsterdam         NLD  Noord-Holland
```

Analyzing DataFrames

Viewing the Data

One of the most used method for getting a quick overview of the DataFrame, is the head() method.

The **head()** method returns the headers and a specified number of rows, starting from the top.

Get a quick overview by printing the first 10 rows of the DataFrame

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.head(10))
```

```
Output:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0     NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
```

```
7    France  48.0  79000.0       Yes
8   Germany  50.0  83000.0        No
9    France  37.0  67000.0       Yes
```

Note: if the number of rows is not specified, the head() method will return the top 5 rows.

Print the first 5 rows of the DataFrame:

```
import pandas as pd
df = pd.read_csv('User_Data.csv')# or excel, json as required
print(df.head())
```

Output:

```
User ID  Gender  Age  EstimatedSalary  Purchased

0  15624510    Male   19            19000          0

1  15810944    Male   35            20000          0

2  15668575  Female   26            43000          0

3  15603246  Female   27            57000          0

4  15804002    Male   19            76000          0
```

There is also a tail() method for viewing the last rows of the DataFrame.

The **tail()** method returns the headers and a specified number of rows, starting from the bottom.

Print the last 5 rows of the DataFrame

```
import pandas as pd
df = pd.read_csv('User_Data.csv')# or excel, json as required
print(df.tail())
```

Output:

	User ID	Gender	Age	EstimatedSalary	Purchased
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

Data wrangling

Data wrangling, also known as data munging or data preparation, is the process of cleaning, structuring, and transforming raw data into a more usable format for analysis and decision-making. It involves various tasks like handling missing values, removing duplicates, standardizing formats, and merging datasets to improve data quality and make it more consumable.

Cleaning Data

Data cleaning means fixing bad data in your data set. Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

Loading dirtydata.csv

```
import pandas as pd
```

```
df = pd.read_csv('dirtydata.csv')
print(df.to_string())#display all rows
```

output:

	Duration	Date	Pulse	Maxpulse	Calories
0	*60*	*'2020/12/01'*	*110*	*130*	*409.1*
1	*60*	*'2020/12/02'*	*117*	*145*	*479.0*
2	*60*	*'2020/12/03'*	*103*	*135*	*340.0*
3	*45*	*'2020/12/04'*	*109*	*175*	*282.4*
4	*45*	*'2020/12/05'*	*117*	*148*	*406.0*
5	*60*	*'2020/12/06'*	*102*	*127*	*300.0*
6	*60*	*'2020/12/07'*	*110*	*136*	*374.0*
7	*450*	*'2020/12/08'*	*104*	*134*	*253.3*

.

.

.

The data set contains some empty cells ("Date" in row 22, and "Calories" in row 18 and 28).

The data set contains wrong format ("Date" in row 26). The data set contains wrong data ("Duration" in row 7). The data set contains duplicates (row 11 and 12). Pandas - Cleaning Empty Cells

Empty Cells(Null values)

Empty cells can potentially give you a wrong result when you analyze data.

```
import pandas as pd
df = pd.read_csv('DATASET/dirtydata.csv')
# identify missing values in each column
print(df.isnull().sum())
# identify missing values in the dataframe
print(df.isnull().sum().sum())
```

```
Output:
Duration    0
Date        1
Pulse       0
Maxpulse    0
Calories    2
dtype: int64
3
```

Remove Rows

One way to deal with empty cells is to remove rows that contain empty cells.This is usually OK, since data sets can be very big, and removing a few rows will not have a big impact on the result. Return a new Data Frame with no empty cells:

Return a new Data Frame with no empty cells:

```
import pandas as pd
df = pd.read_csv('dirtydata.csv')
new_df = df.dropna()
```

```
print(new_df.to_string())
```

Note: By default, the dropna() method returns a new DataFrame, and will not change the original.

If you want to change the original DataFrame, use the **inplace = True** argument

1. Remove all rows with NULL values:

```
import pandas as pd
df = pd.read_csv('dirtydata.csv')
df.dropna(inplace = True)
print(df.to_string())
```

Note: Now, the dropna(inplace = True) will NOT return a new DataFrame, but it will remove all rows containing NULL values from the original DataFrame.

2. Replace Empty Values

Another way of dealing with empty cells is to insert a new value instead. This way you do not have to delete entire rows just because of some empty cells. The fillna() method allows us to replace empty cells with a value:

Replace NULL values with the number 130:

```
import pandas as pd
df = pd.read_csv('dirtydata.csv')
df.fillna(130, inplace = True)
print(df.to_string())
```

3. Replace Only For Specified Columns

The example above replaces all empty cells in the whole Data Frame.

To only replace empty values for one column, specify the column name for the DataFrame:

Replace NULL values in the "Calories" columns with the number 130:

```
import pandas as pd
df = pd.read_csv('dirtydata.csv')
df["Calories"].fillna(130, inplace = True)
print(df.to_string())
```

4. Replace Using Mean, Median, or Mode

A common way to replace empty cells, is to calculate the mean, median or mode value of the column.

Pandas uses the mean() median() and mode() methods to calculate the respective values for a specified column:

Calculate the MEAN, and replace any empty values with it:

```
import pandas as pd
df = pd.read_csv('dirtydata.csv')
x = df["Calories"].mean()
df["Calories"].fillna(x, inplace = True)
print(df.to_string())
```

Calculate the MEDIAN, and replace any empty values with it:

```
import pandas as pd
df = pd.read_csv('dirtydata.csv')
x = df["Calories"].median()
df["Calories"].fillna(x, inplace = True)
print(df.to_string())

#Median = the value in the middle, after you have sorted all values ascending.
```

Calculate the MODE, and replace any empty values with it:

```
import pandas as pd
df = pd.read_csv('dirtydata.csv')
x = df["Calories"].mode()[0]
df["Calories"].fillna(x, inplace = True)
print(df.to_string())
#Mode = the value that appears most frequently.
```

Cleaning Data of Wrong Format

Data of Wrong Format Cells with data of wrong format can make it difficult, or even impossible, to analyze data. To fix it, you have two options: remove the rows, or convert all cells in the columns into the same format.

```
import pandas as pd
# Sample DataFrame
data = {'A': [1, 2, 3, 4], 'B': ['1', '2', '3', '4']}
df = pd.DataFrame(data)
```

```
print(df.dtypes) # Check the original data types
df['B'] = df['B'].astype(int) # Convert column 'B' to integers
print(df.dtypes) # Check the updated data types
```

Output:

```
A     int64
B    object
dtype: object
A    int64
B    int32
dtype: object
```

Convert Date into a Correct Format

In our Data Frame, we have two cells with the wrong format. Check out row 22 and 26, the 'Date' column should be a string that represents a date: Let's try to convert all cells in the 'Date' column into dates.

Pandas has a to_datetime() method for this:

```
import pandas as pd
df=pd.read_csv(r'C:\DataScience\Datasets\dirtydata.csv')
print(df['Date'].dtype)# object means string
df['Date'] = pd.to_datetime(df['Date'])
print(df['Date'].dtype)# datetime64[ns]
```

Removing Rows

The result from the converting in the example above gave us a NaT value, which can be handled as a NULL value, and we can remove the row by using the dropna() method.

Remove rows with a NULL value in the "Date" column:

```
import pandas as pd
df = pd.read_csv(r'C:\DataScience\Datasets\dirtydata.csv')
df['Date'] = pd.to_datetime(df['Date'])
df.dropna(subset=['Date'], inplace = True)
print(df.to_string())
```

Fixing Wrong Data

"Wrong data" does not have to be "empty cells" or "wrong format", it can just be wrong, like if someone registered "199" instead of "1.99". Sometimes you can spot wrong data by looking at the dataset, because you have an expectation of what it should be. If you take a look at our data set, you can see that in row 7, the duration is 450, but for all the other rows the duration is between 30 and 60. It doesn't have to be wrong, but taking in consideration that this is the data set of someone's workout sessions, we conclude with the fact that this person did not work out in 450 minutes.

Replacing Values

One way to fix wrong values is to replace them with something else. In our example, it is most likely a typo, and the value should be "45" instead of "450", and we could just insert "45" in row 7:

Set "Duration" = 45 in row 7:

```
import pandas as pd

df = pd.read_csv(r'C:\DataScience\Datasets\dirtydata.csv')
df.loc[7,'Duration'] = 45
print(df.to_string())
```

For small data sets you might be able to replace the wrong data one by one, but not for big data sets. To replace wrong data for larger data sets you can create some rules, e.g. set some boundaries for legal values, and replace any values that are outside of the boundaries.

Loop through all values in the "Duration" column. If the value is higher than 120, set it to 120:

```
import pandas as pd
df = pd.read_csv(r'C:\DataScience\Datasets\dirtydata.csv')
for x in df.index:
  if df.loc[x, "Duration"] > 120:
    df.loc[x, "Duration"] = 120
print(df)
```

Removing Rows

Another way of handling wrong data is to remove the rows that contains wrong data. This way you do not have to find out what to replace them with, and there is a good chance you do not need them to do your analyses.

Delete rows where "Duration" is higher than 120:

```
import pandas as pd
df = pd.read_csv(r'C:\DataScience\Datasets\dirtydata.csv')
```

```
for x in df.index:
  if df.loc[x, "Duration"] > 120:
    df.drop(x, inplace = True)
# Remember to include the 'inplace = True' argument to make
# the changes in the original DataFrame object instead of
# returning a copy

print(df)
```

Removing Duplicates

Discovering Duplicates: Duplicate rows are rows that have been registered more than one time. By taking a look at our test data set, we can assume that row 11 and 12 are duplicates.

To discover duplicates, we can use the duplicated() method.

The **duplicated()** method returns a Boolean values for each row:

Returns True for every row that is a duplicate, otherwise False:

```
import pandas as pd
df = pd.read_csv(r'C:\DataScience\Datasets\dirtydata.csv')
print(df.duplicated().sum())
```

Output:

1

This shows there is one missing value in the dataset.

Removing Duplicates

To remove duplicates, use the **drop_duplicates()** method.

Remove all duplicates:

```
import pandas as pd
df = pd.read_csv(r'C:\DataScience\Datasets\dirtydata.csv')
df.drop_duplicates(inplace = True)
print(df.to_string())
```

Remember: The (inplace = True) will make sure that the method does NOT return a new DataFrame, but it will remove all duplicates from the original DataFrame.

Renaming columns and rows

The **rename()** method allows you to relabel an axis based on some mapping (a dict or Series) or an arbitrary function.

```
import pandas as pd
import numpy as np
df1=pd.DataFrame(np.random.randn(6,3),\
    columns=['col1','col2','col3'])
print(df1)
print ("After renaming the rows and columns:")
df2=df1.rename(columns={'col1' : 'c1', 'col2' : 'c2'},\
               index = {0 : 'apple', 1 : 'banana', 2 : 'durian'})
print(df2)
```

Output:

```
 col1      col2      col3
0  0.890934  0.753627  1.530386
```

```
1 -0.543602 -0.806000  0.607809
2  1.544517  0.736254  0.065005
3 -0.818535  1.357496  0.774576
4  0.186969 -0.189036  0.700067
5  0.366647 -0.607940 -0.681466
After renaming the rows and columns:
              c1        c2      col3
apple   0.890934  0.753627  1.530386
banana -0.543602 -0.806000  0.607809
durian  1.544517  0.736254  0.065005
3      -0.818535  1.357496  0.774576
4       0.186969 -0.189036  0.700067
5       0.366647 -0.607940 -0.681466
```

Rename multiple columns or rows without much complications

In most of the data you will have issues with spaces or symbols in column names or row names. Changing them directly is an overhead so we have to follow other procedure for less complexity.

```
import pandas as pd
import numpy as np
df1=pd.DataFrame(np.random.randn(6,3),
```

```
                  columns=['A1','B2','C3'])
print(df1)
df1.columns = df1.columns.str.replace(r'\d+','', regex=True)
print(df1)
```

Output:

```
     A1        B2        C3
0  0.042102 -0.061793  0.912059
1  0.428865  0.033813  0.032084
2 -2.025418 -1.869917 -0.736809
3  1.459930 -1.214866  1.806067
4  0.580571 -1.394940  0.283507
5 -1.273295 -1.519599 -1.804755
          A         B         C
0  0.042102 -0.061793  0.912059
1  0.428865  0.033813  0.032084
2 -2.025418 -1.869917 -0.736809
3  1.459930 -1.214866  1.806067
4  0.580571 -1.394940  0.283507
5 -1.273295 -1.519599 -1.804755
```